MKM Research Labs

AI & Agentic Model Governance Module

Unified Specification — Shared Utilities, Two Separate Workflows

Status: Draft v0.4 — for review — supersedes v0.3

Source frameworks: FINOS AI Governance Framework (AIGF) v2.0; Moody's "Model Risk Management in the Age of AI" companion governance paper; existing Moody's Analytics model tiering and validation standards

Prepared for: David Kelly, MKM Research Labs / Kerr Shearer Limited

What changed from v0.3: v0.3 was written as though the module were greenfield. It is not. ModelRisk already carries an AI System record type, an ML Application record type, model chain lineage, a FINOS risk catalogue held as configuration, and committee routing written once and adapted per record type. v0.4 is re-baselined against that code rather than against an imagined starting point, and three consequences run through the document. Classification is carried by the record type, not by a model_class field on a shared record (§2). Two pieces of Part A that v0.3 described as existing — the mechanically-verifiable monitoring library (§3.6) and the change-classification engine (§3.5) — do not exist in the platform and are new build; where v0.3 said "already uses this today" it was describing Moody's process, not platform capability, and Part B's claim to be unchanged rested on that sentence. And the phase plan (§13) now states what is built before stating what each phase adds.

Two sections are new. §3.8 sets out the question library — common questions authored once under an AI/ML Apps grouping, with ML-specific and AI-specific sets beneath, composed onto a record at registration. §3.9 states that the review process is one process across models, ML Apps and AI Systems, and that the reviewer's role is what that sameness protects. Both follow the governing principle this revision makes explicit: the two workflows stay separate, and what they share is utilities, not a merged process.

Two contradictions inherited from v0.2 are also resolved. §6 no longer makes the shared change taxonomy optional for ML Apps when §3.5 and §12 make it mandatory, and the failure-mode register (§9.4) no longer implies a second control register standing alongside the FINOS document-evidencing design.

PART A — SHARED FOUNDATION

---

1. Purpose and Deployment Context

This part covers what ML Apps and AI Systems have in common: the record types that separate them, and the platform utilities both rely on. They are two processes, not one process with a switch in it — what is shared is factored out as a utility and used by each, which is a different thing from a workflow that tries to serve both. Parts B and C then cover what each does differently. The build location is now decided — open-source ModelRisk core, forked into Moody's ModelOne — and §4 sets out what that decision requires of the architecture. Where

a section below describes infrastructure as existing, it means existing in ModelRisk at the version this document is baselined against; §13 says what that is.

2. Model Classification: the SR 26-2 Boundary

SR 26-2 draws an explicit line: generative and agentic AI are excluded from Model Risk Management scope; traditional quantitative models — VaR, IRB PD, logistic regression credit scoring, and the like — remain in it. That line has to be structural rather than a judgment call recorded in a column, and in ModelRisk it already is. The platform keeps separate record types — Model, ML Application, AI System, Product — each with its own commands, projection, field specification and screens. An AI System record cannot be created without naming a foundation model the institution has approved; a Model record has nowhere to put one.

The boundary is therefore the register a record lives in. A misclassification is not a wrong value in a field but a record in the wrong register — which every screen, completeness probe and coverage report already distinguishes, and which no one can produce by mis-clicking a dropdown.

Carrier	Values	Notes
record type	Model · ML Application · AI System	The SR 26-2 boundary itself, and what an exam response or coverage report filters on first. Part B governs the Model and ML Application types; Part C governs the AI System type. Product is a packaging record and is out of scope for this document.
deployment_pattern	Direct prompting · Retrieval-augmented (RAG) · Fine-tuned · Agentic (tool-using) · Multi-agent	Already on the AI System record. Generative-versus-agentic is derived from this, not stored beside it: the last two values are the agentic set. Drives which FINOS risks and which of Moody's nine concepts apply.

v0.3 proposed a model_class enum and a separate ai_system_type field on one shared record. Both are now redundant, and both would be actively harmful. model_class would duplicate the record type, and a duplicated fact is a fact that can disagree with itself. ai_system_type would duplicate deployment_pattern, which exists precisely so that "is anything in the estate agentic" is a query rather than a reading exercise — adding a second field asserting the same thing invites a record marked GENERATIVE whose deployment pattern is Multi-agent, with no rule saying which of the two the FINOS security cluster should believe. Where a report wants a single coarse label, it should derive one from deployment_pattern at read time and store nothing.

2.1 Composition is a lineage relationship, not a classification value

A traditional ML fraud-detection model called as a tool by an agentic system does not need a hybrid classification. It keeps its own ML Application record, governed entirely through the ML Apps workflow (Part B); the agentic system's model chain lineage (§3.4) simply carries a reference to it as an upstream dependency. Classification stays per-record; composition is handled where the platform already handles dependency chains, in the existing model chain module.

3. Shared Utilities (Core Package)

Everything below is used by both workflows without modification, as a utility each calls rather than as a process either belongs to. Nothing in Part B or Part C should reimplement any of it. Each subsection now says whether it exists in ModelRisk today or is new build, because v0.3 did not distinguish the two and the difference accounts for most of the Phase 0 estimate.

3.1 Model inventory and classification

Exists. The record types from §2 already share a classification vocabulary: tier, materiality, complexity and their rationales are named identically on the Model, ML Application and AI System records. That naming is what lets one completeness probe or one coverage report count across all three without special-casing any of them, and it is the reason a shared field for the class itself would add nothing.

3.2 Event log and bi-temporal stamping

Exists. The existing immutable, hash-chained event log and bi-temporal stamping apply to every event and versioned artifact produced by either workflow — validation sign-offs and recertifications for ML Apps; configuration snapshots, envelopes, and trajectory scores for AI Systems.

3.3 Committee coverage model

Exists; routing inputs to extend. Both classes route through the existing Company MRC / division MRC structure, and the routing is already written once and adapted per record type rather than copied — model, product, handoff, ML Application and AI System all travel the same committee machinery through a per-record adapter. An AI System's rating is proposed by its owner and does not move until the committee approves it, exactly as a model's does. What this spec adds is record type and, for AI Systems, deployment pattern and risk tier as further routing inputs alongside the existing materiality and mission-criticality fields — not a parallel committee track.

3.4 Model chain lineage

Exists; node types to extend. Lineage nodes can now be MODEL, ML_APPLICATION or AI_SYSTEM, and an AI System's harness can reference an ML App as a tool dependency (§2.1). Foundation models and MCP servers (AI Systems only) also appear as lineage nodes, per the original FINOS-only spec.

3.5 Change-classification engine

New build. Moody's four-tier change taxonomy — Administrative, Bounded, Measurement, Material — becomes a shared engine. v0.3 described this as "generalised" from something already in place; nothing of the kind exists in ModelRisk, which today records a change as an event against a record and expresses materiality through review status, validation status and recertification rather than through a change class. Building it is Phase 0 work and should be estimated as such.

Both workflows then classify every proposed change through the same engine, with the revalidation depth each tier calls for interpreted per workflow — a Material change to an ML App triggers the existing validation-pack refresh; a Material change to an AI System triggers re-baselining per §9.5. ML Apps adopt it in full, replacing their existing change-management triggers rather than running alongside them (§12).

3.6 Mechanically-verifiable-field monitoring library

New build, and the largest single item in Phase 0. Numeric, categorical and ordinal output fields should be monitored the same way regardless of what produced them: backtesting against realised outcomes, distribution and transition-matrix monitoring, rank correlation, population stability index.

v0.3 said "ML Apps already uses this today", and both Part B's claim to be unchanged and §9.3's claim to reuse an identical library rested on that sentence. It is not true of the platform. ModelRisk's monitoring module produces an Ongoing Monitoring Report — one model's monitoring position, its stated approach, its dates and its RAG status — and computes none of these statistics. The techniques are Moody's existing practice; they are not yet

shared infrastructure that Part C can pick up. Whoever estimates Phase 0 should read this subsection as a build, not as a wiring exercise.

Once it exists, AI Systems reuse it for their mechanically-verifiable output fields (per Moody's Table 2) rather than growing a second implementation — only free-form text and trajectories need AI-specific monitoring (ACE, trajectory scoring; §9.3).

3.7 Risk / mitigation mapping engine

Partly exists. A generic engine that accepts any risk taxonomy as configuration. ModelRisk already carries the FINOS AIGF catalogue this way and evidences its mitigations from documents tagged against a record; the same mechanism serves ISO 31000 / COSO clause coverage. What is missing is the SR 11-7 / SR 26-2 validation criteria as a second data set for ML Apps. One engine, two data sets — not two engines, and not a second evidencing mechanism (§9.4).

3.8 Question library: common questions once, specific questions per workflow

ML Apps and AI Systems are two processes, not one process with a switch in it. What they share is factored out and called by each; it is not a merged workflow that tries to serve both. The questionnaire is the clearest case, because today it is the least shared thing in the platform and parts of it are maintained twice.

Admin currently carries three unrelated question lists — review questions for a model, ML Application questions, and AI System questions — each authored and seeded independently, and each copied onto a record as its own snapshot at registration. That separation is mostly right, and deliberately so: not one of the ML questions is answerable about a foundation model nobody in the firm trained, and the AI set is written against the FINOS catalogue so that answering it is what moves those rows.

But a handful of questions are the same question, kept in two places. Use Case is asked of both — "What decision does this model inform, and what is it explicitly not to be used for?" against "What decision or task does this system support, and what is it explicitly not to be used for?" — and so is Explainability. Those are not two questions that happen to resemble each other. They are one question whose wording has already drifted, which is what duplication does before anyone notices it has happened.

The structure is therefore three lists, not two and not one:

List	Where it sits in Admin	What it holds
Common	Under an AI/ML Apps grouping, above both	What is asked of any application of a model, generative or not: use case and exclusions, explainability to an affected person, human oversight, who could be disadvantaged and how you would know, and what would show it had stopped working. Authored once.
ML Apps	Beneath the common list	Everything that presumes the firm fitted the thing: training data and its representativeness, data leakage, out-of-sample performance, benchmarking against a simple baseline, overfitting and stability.
AI Apps	Beneath the common list	Everything that presumes it did not: model provenance, data egress and its contractual basis, grounding, hallucination exposure, determinism, prompt and input integrity, agency and authority, regulatory classification.

Composition happens at registration, not at display. A record's questionnaire is still one flat snapshot on the record — the common questions followed by its own — so nothing downstream needs to know the list was

assembled from two sources. Review, the completeness probes and the sign-off pack all read what they read today. This preserves the copy-at-registration semantics all three lists already have, and it means editing a common question changes what future records are asked without rewriting the questionnaire of any record already under review.

One consequence to plan for: moving a question into the common list is a content migration, not only a configuration change. Records registered before the move keep the wording they were registered with, which is the correct behaviour — a review answers the question it was actually asked — but it means the common list should be settled before Phase 1 seeds anything, rather than being tidied afterwards.

3.9 One review process; the reviewer's role is the constant

The review of an ML App or an AI System looks and behaves like a model review. This is not a styling preference — it is what makes the reviewer's role portable, and the role is the point. A reviewer who has done a model review should be able to review an AI System without learning a second screen or a second vocabulary, because what is being asked of them is the same thing: read the evidence, challenge it, and put a name to the conclusion.

Most of this already holds and needs protecting rather than building. The seven-stage sequence — preliminary documents, assumptions, questions, weaknesses, limitations of use, monitoring approach, remediation — is shared, and the committee routing is written once and adapted per record type (§3.3). The independence rule is shared too: the reviewer is named from Admin > People, and a record whose reviewer is its owner is a completeness finding on every record type equally.

Three things follow, and they constrain Parts B and C as those grow:

- The stage sequence does not fork. An AI System's Establish Baseline (§9.2) and Operate & Measure (§9.3) are lifecycle phases, not review stages: they produce evidence that the questions stage reads. Neither adds an eighth stage, and no record type gets a stage the others lack.
- The reviewer is a role, not a record type. The same person, the same independence check, and the same challenge thread — which is evidence, and therefore append-only — regardless of what is under review.
- The questions stage is the only per-type variation, and its label has to follow the record rather than be fixed. It reads "the nine questions" today because that is what a model is asked; with a composed questionnaire (§3.8) the count differs by record type, so the label should derive from the questionnaire the record actually carries.

Where an AI System needs something a model review has no equivalent of — a variability envelope to judge a trigger against, a trajectory score — it enters through the evidence a stage reads, not through a new stage. The reviewer sees a familiar shape with unfamiliar content in it, which is the right way round.

4. Build Once, Configure Twice

Decided: the module is built once in open-source ModelRisk, with Moody's ModelOne as a downstream fork. The relationship is one-way — ModelOne consumes the core package; there is no contribution path back upstream. That decision raises the stakes on the governing constraint stated once here and assumed everywhere else in this document: anything likely to diverge between deployments — branding, thresholds, committee routing, jurisdictional regime data, judge model selection — must live in configuration, not code. With a two-way relationship, logic that leaked into the fork could eventually be reconciled by pulling it back. With a one-way fork, any logic that leaks into ModelOne instead of staying in config is permanent divergence from day one: it

will never be reviewed against the core package again, and every future core-package fix has to be manually re-applied to it. The core package ships as a versioned, fully tested Python package with a stable public API; ModelOne imports it and supplies configuration and a thin integration layer only. This is not a preference — it is the only way the fork stays maintainable given no contribution path back. One practical note: the constraint is enforceable rather than aspirational. ModelRisk's own coding rules already gate it — parameters live in the config package and never hardcoded in source (R1), with the theme rule (R10) as the worked precedent for pushing a whole category of divergence into configuration and then auditing it. Anything this document identifies as deployment-varying should be added to that audit as it is built, so the fork's configuration boundary is checked mechanically on every commit rather than at review time.

PART B — ML APPS WORKFLOW

5. Scope and Existing Basis

ML Apps covers everything SR 26-2 leaves inside Model Risk Management scope: traditional quantitative and machine-learning models with no generative or agentic component. This workflow formalises what already operates rather than proposing something new — the existing Core Engines / Analytics and Science Models / Product and Deployment Models tiering, the existing validation template structure, and the existing MRC artefact set (model database, ToR, minutes, monitoring reports, recertification, annual audit) all carry over unchanged. What this spec adds is the wiring into the shared utilities in Part A — classification, lineage and the shared event log — plus three changes that are more than wiring and should not be described as such: ML Apps adopt the shared change taxonomy in place of their existing triggers (§3.5, §12); the monitoring library that Part B's "unchanged" rests on has to be built before anything can be wired to it (§3.6); and the questions common to both workflows move into the shared list, leaving the ML-specific set behind (§3.8).

6. ML Apps Lifecycle

Stage	Content
Build & document	Existing model development documentation standard: rationale, approach, assumptions, data sources, testing approach — unchanged.
Validate	Existing expert/outcome-based validation criteria (conceptual soundness, data quality, methodology, testing) — unchanged, and run through the same review process and reviewer role as every other record type (§3.9).
Monitor	Uses the shared mechanically-verifiable-field library (§3.6) — PSI, backtesting, transition-matrix monitoring against the model's existing monitoring reports. That library is Phase 0 build, not an existing dependency; until it lands this stage is unchanged from today.
Periodic review & recertification	Existing annual recertification and R/A/G status process — unchanged, now logged through the shared event log (§3.2).
Material change	Adopts the shared four-tier change-classification engine (§3.5) in full, replacing the existing SR 11-7 / SR 26-2–aligned triggers rather than running alongside them. v0.3's table made this optional and so contradicted its own §3.5 and §12; it is mandatory.

7. What Does Not Apply to ML Apps

Explicitly out of scope for this workflow, and reserved for AI Systems only:

- Backbone / harness separation — there is no third-party-hosted, opaque generative backbone to separate from the institution's own logic.
- Model pinning against a provider-managed snapshot, sampling discipline (temperature, top-p), instruction hierarchy.
- Variability envelope — ML Apps are deterministic given fixed inputs; there is no run-to-run stochasticity to baseline.
- Anchored Continuous Evaluation and trajectory governance — no free-form output or multi-step agentic path to score.
- The standing AI risk register sourced from FINOS (bias, IP, reputational risk, deep supply-chain compromise; §10). Where ML Apps already has its own bias / fair-lending controls, they remain in the existing validation standard and are not duplicated here.

PART C — AI SYSTEMS WORKFLOW

---

8. Scope

AI Systems covers generative and agentic AI — explicitly excluded from SR 26-2 MRM scope, but governed under this framework regardless, consistent with the Policy's own AI carve-out logic, and squarely within PRA SS1/23 scope where UK/EU operations apply. This is the workflow the original FINOS-only spec and the first unified spec both described; it carries over here largely unchanged, now explicitly scoped away from ML Apps rather than sharing a taxonomy with it.

9. Five-Phase Lifecycle

9.1 Design & Instrument

A configuration snapshot captures model pin, output schema, sampling parameters, and instruction hierarchy as a single versioned artifact. Trace capture depth follows the single-call / workflow / agent taxonomy. Absorbs FINOS RI-5 (foundation model versioning) and RI-6 (non-determinism).

9.2 Establish Baseline

An expert-grounded benchmark (dataset, rubric, judge) plus a variability envelope — N repeated runs under the pinned configuration, stored as percentile-based limits.

9.3 Operate & Measure

Anchored Continuous Evaluation (ACE) for free-form fields; the shared mechanically-verifiable library (§3.6) for numeric/categorical/ordinal fields, which this phase depends on and which does not exist yet; trajectory scoring

via graph-edit distance for agentic systems. Absorbs FINOS RI-19 (data quality and drift).

9.4 Escalate

A trigger register calibrated against the variability envelope, and a failure-mode register whose entries are identified by a FINOS risk/mitigation pair, covering both Moody's operational failure modes and FINOS's agentic security cluster (tool chain manipulation, MCP server compromise, agent state persistence poisoning, credential harvesting).

The register does not carry its own mitigation status. ModelRisk's FINOS work already answers "is this mitigation in place" from documents tagged against the record, and a status column here would be a second control register answering the same question from a different source — which is how two registers come to disagree and neither is trusted afterwards. A failure-mode entry therefore holds only what is specific to it: the risk and mitigation identifiers, the trigger that detects the failure, and the fallback state to enter when it fires. Whether the mitigation is evidenced is resolved at read time from the tagged documents, exactly as the FINOS coverage view resolves it now.

Trigger thresholds are set per property, not uniformly across the four: a Tier 1 model's bar on correctness, consistency, robustness, and predictability is calibrated individually to what the specific use case demands, rather than one severity level applied to all four. A customer-facing generative system, for instance, may warrant a materially tighter robustness threshold (adversarial resistance) than its predictability threshold (confidence calibration), depending on where the actual exposure sits. This means each trigger register entry carries a threshold per property per model, not a single per-model severity tier — the model owner and escalation chain set each independently at baseline, and materiality/mission-criticality tiering informs the starting point for each property rather than fixing all four at once.

9.5 Revalidate

An immutable release manifest and Moody's four-tier change classification (§3.5), applied here as the primary use of that engine. A FINOS taxonomy version bump is treated as a Measurement-class change against the mapping engine.

9.6 Judge Model Governance

The judge used for ACE (§9.3) and for scoring agentic trajectories is not a static, separately-approved configuration parameter. It is itself a model, and is governed exactly like any other model in this platform: it goes through the same onboarding, classification, and committee approval as any AI System, is subject to its own configuration snapshot and revalidation regime, and a change to it — a version bump, a re-prompted rubric, a swapped provider — is a Measurement-class change under §3.5, requiring revalidation of the evaluation stack before its scores are trusted again, consistent with Concept 9's treatment of judge and rubric changes.

Practically: a judge model has its own model_id and its own model_class = AI_SYSTEM record, distinct from the model_id of whatever it is scoring. Its lineage record (§3.4) marks it as a dependency of every model whose ACE or trajectory monitoring it feeds, so a judge-model change surfaces in the affected-artifacts list of every model that depends on it rather than needing to be discovered separately. ModelRisk today treats a judge as an assessment kind — LLM-as-a-judge is one of the values an assessment can take, and a recorded judge evaluation is what the completeness framework reads as testing evidence. That evidence model is right and stays. What changes is that the assessment names the judge's own record rather than describing it in prose, so the dependency becomes a lineage edge instead of a sentence.

10. Standing Risk Register

Bias and discrimination (RI-16), IP and copyright (RI-23), reputational risk (RI-20), and the deeper supply-chain risks Moody's only treats generically (RI-7, RI-8, RI-26) sit outside the five-phase lifecycle as a standing register, sourced from FINOS, reviewed at every Material revalidation and at onboarding. This register is AI Systems–only by design — see §7 for the corresponding ML Apps boundary. Visibility is identical across the ModelRisk and ModelOne deployments: no entries are muted or filtered per deployment, so the register reads the same whether viewed from MKM's own instance or Moody's fork.

11. Illustrative Data Model

Grouped by what's shared versus what belongs to a single workflow.

Shared (Part A)

```
# Classification is the record type. There is no model_class field:
# Model, MLApplication and AISystem are separate records that share
# these field names, which is what lets one probe count across all three.

class SharedClassification:      # named identically on all three records
    tier: str
    materiality: str
    complexity: str

class LineageNode:
    record_id: str
    node_type: str               # MODEL | ML_APPLICATION | AI_SYSTEM
                                 # | FOUNDATION_MODEL | MCP_SERVER
    upstream_of: list[str]

class ChangeRecord:              # new build, see 3.5
    change_class: ChangeClass    # ADMINISTRATIVE | BOUNDED | MEASUREMENT | MATERIAL
    revalidation_depth: str
    affected_artifacts: list[str]

class QuestionList:              # see 3.8
    scope: str                   # COMMON | ML_APP | AI_SYSTEM
    questions: list[Question]    # COMMON + own scope, composed at registration
```

ML Apps only (Part B)

```
class ValidationPack:
    model_id: str
    conceptual_soundness_review: str
    data_quality_review: str
    testing_results: dict

class RecertificationRecord:
    model_id: str
    period: str
    rag_status: str                  # Red | Amber | Green
```

AI Systems only (Part C)

```
class ConfigurationSnapshot:
    model_id: str
    model_pin: FoundationModelRef
    output_schema: dict
    sampling_params: dict
    instruction_hierarchy: list[str]

class VariabilityEnvelope:
    config_version: int
    metric: str
    percentiles: dict[float, float]

class FailureModeRegisterEntry:
    finos_risk_id: str | None
    finos_mitigation_id: str | None
    trigger_id: str | None
    fallback_state: str
    # no mitigation_status —— resolved from tagged documents (§9.4)

class StandingRiskRegisterEntry:
    finos_risk_id: str              # RI-16, RI-20, RI-23, etc.
    applicable: bool
    # mitigation_status is derived, not stored (§9.4)
```

12. Decision Log

Six of the seven decisions raised in v0.2 were resolved in v0.3. v0.4 adds four more, three of which arose from baselining this document against the code and one from the requirement that the two workflows stay separate while their common parts become utilities. All are written into the relevant sections above; this log records what was decided and where it lives, so the reasoning doesn't have to be reconstructed later.

Decision	Resolution	Where it lives
ML Apps change classification	Adopts the shared four-tier taxonomy fully, replacing its existing triggers	§3.5, §6
Build location	Open-source ModelRisk core, forked into Moody's ModelOne	§4
Fork relationship	One-way — ModelOne consumes the core package only, no contribution path back	§4
Standing risk register visibility	Identical across both deployments; no entries muted	§10
Judge model approval	Not a separate config decision — the judge is itself a governed model, approved via MRC like any other	§9.6
Property tiering	Bar varies by property (correctness, consistency, robustness, predictability) depending on use case, not a uniform bar per materiality tier	§9.4
Classification carrier (v0.4)	The record type, not a model_class field; generative versus agentic is derived from deployment_pattern rather than stored a second time	§2
Part A build status (v0.4)	The monitoring library and the change-classification engine are new build, not existing infrastructure; v0.3 described both as already in place	§3.5, §3.6

Decision	Resolution	Where it lives
Question library (v0.4)	Common questions authored once under an AI/ML Apps grouping, with ML-specific and AI-specific lists beneath, composed onto the record at registration	§3.8
Mitigation status (v0.4)	One source of truth: the failure-mode register holds trigger and fallback only, and mitigation status resolves from tagged documents	§9.4

Still open

- Refresh cadence for the FINOS taxonomy as configuration data — manual quarterly review, live query via the FINOS AIGF MCP Server, or a hybrid. No default assumed; flagged for a decision before Phase 2 (§13) begins, since it affects how the configuration layer is built, not just how it's maintained.

13. Phased Implementation Plan

Baselined against ModelRisk as it stands, not against a blank sheet. Already built, and therefore in no phase below: the AI System record type, carrying foundation model, pinned version, version policy, hosting, deployment pattern, data egress, EU AI Act class, human oversight, guardrails, grounding and output variability; the ML Application record type with its training and review-policy surfaces; model chain lineage; the immutable event log; MRC routing written once and adapted per record type; the FINOS risk catalogue as configuration with document-based mitigation evidencing; and the completeness audit that reads across record types. Phase 0 is what remains of Part A — less than v0.3 implied in some places and considerably more in others.

Phase	Scope
0	What remains of the shared core (Part A): the change-classification engine (§3.5) and the mechanically-verifiable monitoring library (§3.6), both new build and the bulk of the phase; committee routing extended to take record type and deployment pattern; lineage node types extended for foundation models and MCP servers; the SR 11-7 / SR 26-2 criteria added as a second data set to the existing risk-mapping engine. Fully tested.
1	ML Apps wiring: connect existing validation, monitoring and recertification to the shared core; migrate the existing change triggers onto the four-tier engine; extract the common questions into the shared list (§3.8) before anything further is seeded. The workflow itself is largely unchanged, but the last two are migrations, not wiring, and the question split has to settle first because records keep the wording they were registered with.
2	AI Systems configuration schema and regulatory regime map, defined as data against the shared core's config contract. Blocked until the FINOS refresh cadence is decided — see Still open above, which is why that decision is worth taking early.
3	AI Systems Design & Instrument / Establish Baseline (Concepts 1–4). The configuration snapshot is partly present as current-value fields on the AI System record — model pin and version policy are there, sampling parameters are not — so this phase turns them into a versioned artifact rather than creating them. The variability envelope is new.
4	AI Systems Operate & Measure / Escalate (Concepts 5–8), including the failure-mode register populated from FINOS — identity, trigger and fallback only, with no status column of its own (§9.4).
5	AI Systems Revalidate (Concept 9) and standing risk register (§10).
6	Coverage reporting across both workflows, using the record type as the primary filter.

14. References

FINOS AI Governance Framework v2.0 (October 2025): air-governance-framework.finos.org

Moody's, "Model Risk Management in the Age of AI" — companion governance paper, internal PDF

Grevenbrock, N., Zhao, Z., & Patel, N. (2026). Model Risk Management in the Age of AI. SSRN: https://ssrn.com/abstract=6859138

SR 26-2 / OCC Bulletin 2026-13 (revised interagency guidance on MRM, effective 17 April 2026)

PRA SS1/23, Model Risk Management Principles for Banks (April 2026 update)

EU AI Act, Regulation (EU) 2024/1689

NIST AI 100-1 (AI RMF 1.0); NIST AI 600-1 (Generative AI Profile)

These citations are carried over from v0.3 and have not been checked against the issuing bodies' published texts. The SR 26-2 / OCC 2026-13 effective date, the SS1/23 update date and the AIGF version number should each be verified before this document goes outside MKM.